Lab 6: Working with GPIO Pins

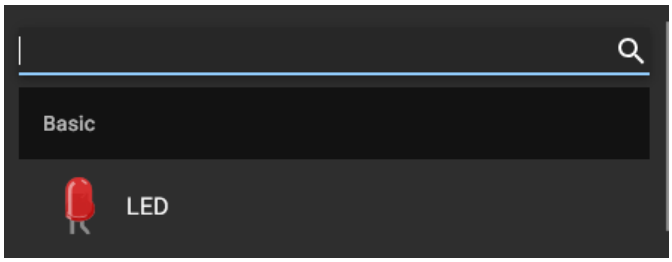
In this lab, we'll work with the ***DDR*** (data direction) and ***PORT*** (output data value) registers, which will allow our code to interact with the outside world. In this case, we'll implement a ***binary display*** – a type of display meant to visualize a binary value that was common on old computers.

Binary Sequence Displays

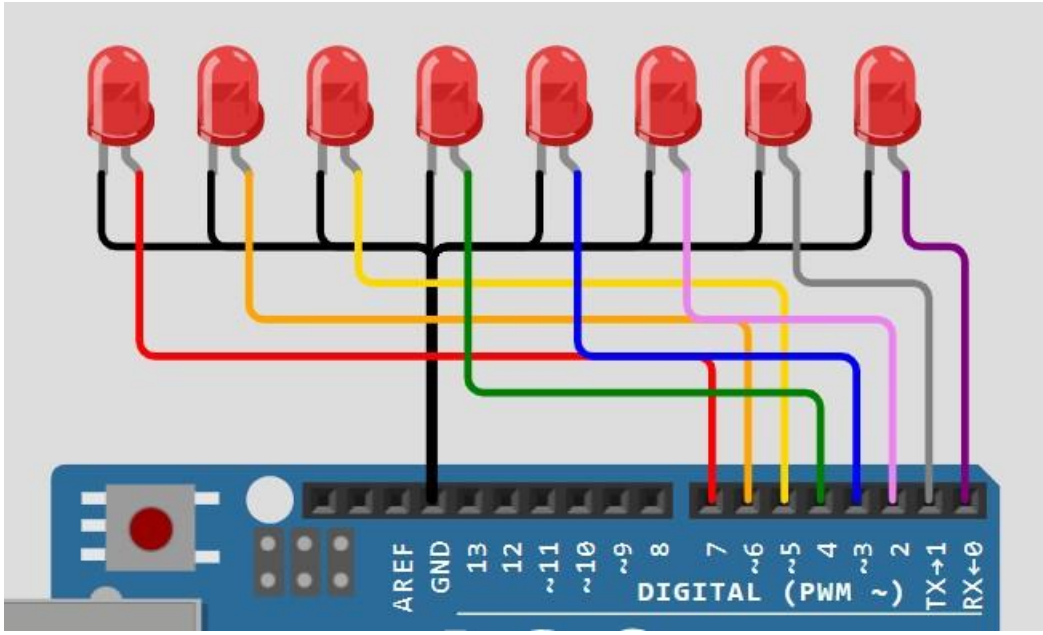
On old machines, such as the classic Altair 8800 (the machine that helped move computing from dedicated computing rooms to offices and labs), there was no graphical display. Instead, a sequence of ***LEDs*** was provided to show the current ***binary state*** of a given memory space. This was particularly useful on machines like the Altair, as they had to be programmed – one instruction at a time – manually with switches. The binary display then aided the programmers in checking their code input and verifying the output of the machine.

To make our own, we'll need ***8 LEDs***, one for each bit, allowing us to display a full byte of information at a time. To make things easier, we'll wire ***all 8*** of these LEDs to the ***"D"*** block of pins. If you check the slide documentation, you'll see that the D block – pins ***PD0*** through ***PD7***, is the only block that offers a full 8 pins (both the C and B blocks only offer 6 pins each, so we'd have to split the display across them otherwise).

You can find the LED under the + symbol button in the Wokwi editor. The standard LED color is red by default, but it can be changed after placing the LED. You'll want to place a total of 8.



Each LED in the ***Wokwi*** editor should have the ***straight leg*** (the ***cathode***, or ***negative lead***) connected to the ***GND*** pin of the Arduino. Then, the ***bent leg*** of each LED should be connected to a consecutive pin on the Arduino. Click on a ***pin*** or ***LED leg*** to start a wire connection. An image is provided below that demonstrates the wiring and placement of the LEDs.



Note that they all share a ***GND***, but each has it's own pin to ***power*** the LED. Also note that we aren't using any resistors, which we would want to do in a real-world situation to avoid burning out our GPIO pins.

Because we're using ***PD0-PD7*** in order, we can ***output*** a binary value by simply setting the ***PORTD*** registry to whatever the binary value is. This will turn on the LED wherever there is a ***1*** in the 8-bit number and turn off the LED wherever there is a ***0*** in the number. If we look at the ***Atmega328P*** datasheet, we can see the addresses of the ***DDRD*** and ***PORTD*** registers that we need to modify – ***0x0B*** for the ***PORTD*** register and ***0x0A*** for the ***DDRD*** register:

13.4.8 PORTD – The Port D Data Register

Bit	7	6	5	4	3	2	1	0	
0x0B (0x2B)	PORTD7	PORTD6	PORTD5	PORTD4	PORTD3	PORTD2	PORTD1	PORTD0	PORTD
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

13.4.9 DDRD – The Port D Data Direction Register

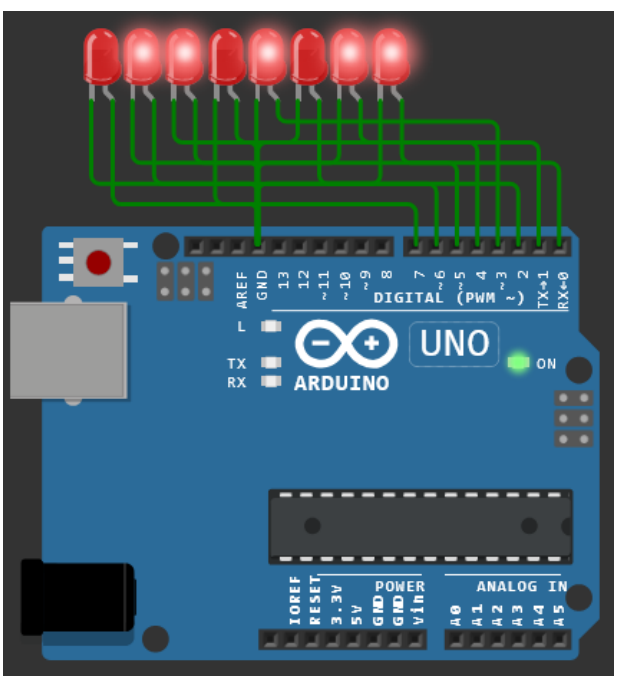
Bit	7	6	5	4	3	2	1	0	
0x0A (0x2A)	DDD7	DDD6	DDD5	DDD4	DDD3	DDD2	DDD1	DDD0	DDRD
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

To display a given number in binary using the LEDs, we just need to write that number to the ***DDRD*** and ***PORTD*** registers. Here's some simple example code to do that (C/C++ on left, assembly code on the right):

```cpp
extern "C" {
  void display(void);
  extern uint8_t val_to_display;
}

void setup() {
  // Set the display value and display
  val_to_display = 107;
  display();
}

void loop() {
  // Nothing here
}
```

```asm
.global display
.global val_to_display

.data
val_to_display: .BYTE 0

.text
display:
push r20
lds r20, val_to_display
out 0x0A, r20
out 0x0B, r20
pop r20
ret
```

In this code, we set up a variable, ***val_to_display***, and a subroutine, ***display***. The display subroutine reads the ***val_to_display*** variable into a register, and then uses the ***OUT*** instruction to write that value to the I/O registers, specifically ***0x0A*** (DDRD) and ***0x0B*** (PORTD). Any ***bit*** in the ***val_to_display*** number that is ***true*** will set that GPIO pin ***direction*** to ***1*** (output) and output value to ***1*** (on), which will turn on the LED of the associated pin. In this example, I used ***107*** as the input number:



We can see the LEDs are lit in the following pattern:

Off	***On***	***On***	***Off***	***On***	***Off***	***On***	***On***
0	***1***	***1***	***0***	***1***	***0***	***1***	***1***

Doing the binary-to-decimal conversion:

$$64 + 32 + 8 + 2 + 1 = 107$$

If this demo code isn't working properly for you, double check the wiring of your LEDs – they should be connected in order, left-to-right, to pins 7 through 0 on the Arduino, and the other leg of each LED should be tied to GND.

```cpp
extern "C" {
    void display(void);
    extern uint8_t val_to_display;
}

void setup() {
    // Initialize display value
    val_to_display = 0;
}

void loop() {
    // Call display function and wait
    display();
    delay(1000);
}
```

```asm
.global display
.global val_to_display

.data
val_to_display: .BYTE 0

.text
display:
push r20
lds r20, val_to_display
inc r20
out 0x0A, r20
out 0x0B, r20
sts val_to_display, r20
pop r20
ret
```

With a minor modification to the ***code***, we can turn this into a binary counter that shows us the number of seconds that have passed. This code initializes the value to display to ***0*** in the ***setup()*** function, and then repeatedly calls the display subroutine with a 1-second ***delay*** inside the ***loop()*** function.

In our assembly code, we load the ***val_to_display*** as before, but we then ***increment*** it by one using the ***INC*** instruction, and then before we're finished we write the incremented value back to the ***val_to_display*** variable using ***STS***.

The result will be an LED counter that slowly counts upwards in binary from 0 to 255. Once it hits 255, it will roll over to zero and start all over again. You can modify the ***delay*** time to make the timer cycle faster or slower.

Submittable Assignment – GPIO Lightshow

Of course, one of the first things many people did with these kinds of old binary displays was find ways to have fun by displaying various patterns. For your submittable assignment, you must create a small GPIO light show:

- It should have ***at least three*** distinct "patterns", each of which should have multiple "frames" of animation or LED changes. For example, the counter example above counting from 0 to 255 would be one pattern, with 255 distinct "frames".
- Most of your logic should be in your ***assembly*** file – you can use the ***C/C++*** code to handle ***delays*** and initialize variables. You should use ***branches***, ***jumps***, or ***subroutines*** to implement cycling between the different animation types

Some potentially useful ideas on patterns and the mathematical operations that can produce them:

- Increment/Decrement a value until rollover (i.e. the counter example)
- Use the ***ASR*** (arithmetic shift right) on the starting value ***128*** to create a display that fills up from left-to-right one LED at a time (128 shifted right → 64 → 32 → 16 etc.)
- Toggling the ***DDRD/PORTD*** value between ***0*** and ***255*** to create a flashing display
- Toggling between 170 (10101010) and 85 (01010101) to create an alternate flashing between even and odd LEDs
- Use the ***LSL*** (logical shift left) on the starting value ***1*** to create a single chasing LED that moves from right to left (1 → 2 → 4 → 8 → 16 → 32 etc.). You can also use the ***LSR*** (logical shift right) on the starting value ***128*** to chase the LED from left-to-right or combine them to create an LED that bounces from one side to the other.
- Use ***SWAP*** on a value to swap its nibbles (switching the first 4 and the last 4 bits). This will swap the behavior of the first four and the last four LEDs – i.e. a value of 00001111 would become 11110000.
- Use combinations of these individual functions, i.e. increment a value, and then SWAP its nibbles on each frame, which could lead to interesting effects and crossovers.

Lab Submission

When finished, save your GPIO lightshow code into the following two plain text files:

- ***Lab6_display.ino*** (C/C++)
- ***Lab6_display.S*** (Assembly)

I recommend just copy-pasting all the code from the ***Wokwi*** editor into new files in ***Notepad++*** or a similar program. Don't forget to include ***comments*** on your code and include your ***name*** and ***student number*** at the top. When you've done that, submit your code on Moodle for marking.